TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 3

ReplicaSets and Deployments

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 3: ReplicaSets and Deployments 3

3.1 From pods to ReplicaSets 3

3.2 Rolling updates 4

3.3 Rollout history and rollback 5

3.4 Release strategies 6

3.5 Live lab: deploy, update, break, roll back 6

Class 3 - Top 20 interview questions 8

K8S · CLASS 03 · 60-90 minutes

ReplicaSets and Deployments

By the end you can deploy, scale, update and roll back an application with zero downtime, and explain blue-green and canary releases.

What you will learn today

- ReplicaSet - keeping N copies alive
- Deployment - the object you use every day
- Scaling up and down
- Rolling updates: maxSurge and maxUnavailable; the Recreate strategy
- Rollout history, change-cause and rollback
- Release strategies: blue-green and canary
- Lab: upgrade nginx, break it with a bad image, roll back

Part	Topic	Time
1	Story, ReplicaSet and Deployment	15 min
2	Rolling update and rollback	20 min
3	Blue-green and canary	10 min
4	Live lab	25 min
5	Recap and interview questions	10 min

3.1 From pods to ReplicaSets

Ravi's story

Ravi's lunch-time rush needs 3 copies of the menu service. He created 3 pods by hand. At 1 PM a node rebooted and two pods vanished - nothing brought them back. Worse, releasing a new menu version meant deleting and recreating pods, and customers saw errors. He needed something that *keeps* 3 copies alive and swaps versions safely.

A **ReplicaSet** ensures that a specified number of identical pods are running at all times. It uses a label selector to count its pods and a pod template to create new ones.

A **Deployment** manages ReplicaSets for you and adds versioned updates and rollback. You almost never create a ReplicaSet directly - you create a Deployment.

Everyday example

A restaurant manager is told "always keep 3 cooks on the shift". If one goes home sick, the manager calls in a replacement (ReplicaSet). When the restaurant introduces a new uniform, the manager swaps cooks *one at a time* so the kitchen never closes, and keeps the old uniforms in case customers complain (Deployment with rollback).

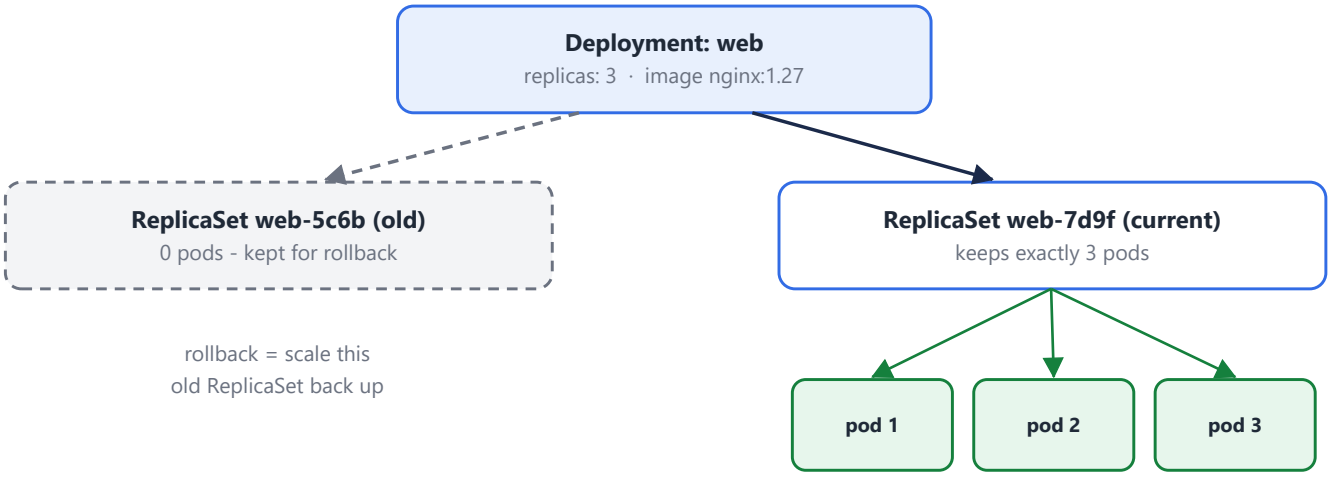


Figure 3.1 - A Deployment owns ReplicaSets, which own Pods

deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  revisionHistoryLimit: 10        # old ReplicaSets kept for rollback
  selector:
    matchLabels:
      app: web                    # must match the template labels
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                 # at most 1 extra pod during update
      maxUnavailable: 0           # never go below 3 ready pods
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.26
          ports:
            - containerPort: 80
```

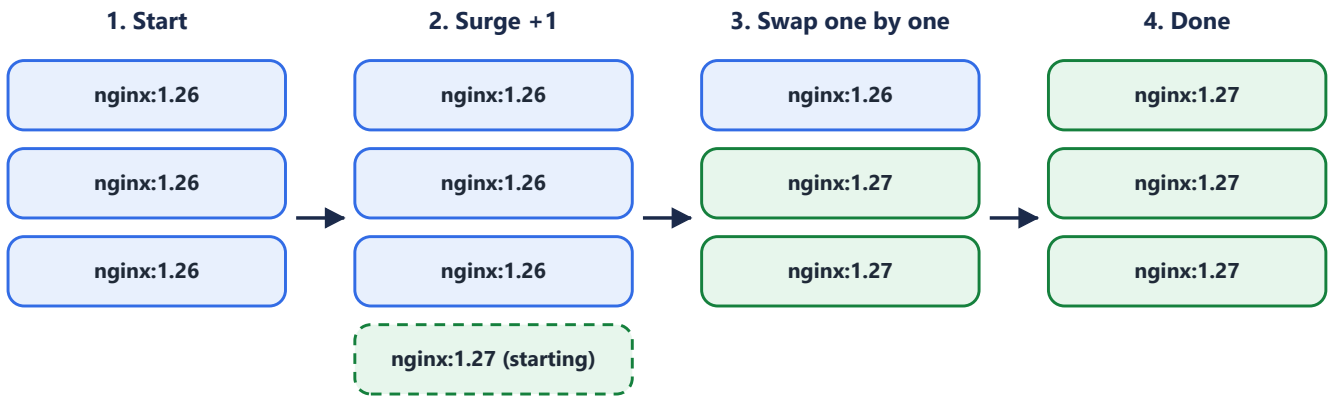
Common mistake
The `selector.matchLabels` must match `template.metadata.labels`. If they do not, the API server rejects the Deployment. The selector cannot be changed after creation.

3.2 Rolling updates

When you change the pod template (for example the image), the Deployment creates a **new ReplicaSet** and gradually moves pods from old to new. Two settings control the speed:

Setting	Meaning	Default
maxSurge	How many pods above `replicas` may exist during the update	25%

Setting	Meaning	Default
maxUnavailable	How many pods below `replicas` may be unavailable during the update	25%



maxSurge: 1 (one extra pod allowed) · maxUnavailable: 0 (never below 3 ready)

Figure 3.2 - Rolling update from nginx:1.26 to nginx:1.27 with maxSurge 1, maxUnavailable 0

The other strategy, **Recreate**, kills all old pods first and then starts new ones. It causes downtime but is needed when two versions cannot run side by side (for example a database schema that only one version understands).

> **Everyday example**
> Changing the tyres of a bus while passengers ride: you replace one tyre at a time (rolling update). Recreate is parking the bus at a garage, removing all tyres, then fitting new ones - simpler, but nobody travels in the meantime.

3.3 Rollout history and rollback

Each template change creates a **revision**. Old ReplicaSets are kept (scaled to 0), so rolling back is fast - Kubernetes just scales the old ReplicaSet up again.

Rollout commands

```
kubectl rollout status deployment/web
kubectl rollout history deployment/web
kubectl rollout history deployment/web --revision=2
kubectl rollout undo deployment/web                  # previous revision
kubectl rollout undo deployment/web --to-revision=1
kubectl rollout pause deployment/web                 # batch several changes
kubectl rollout resume deployment/web
kubectl rollout restart deployment/web               # restart pods, same spec
```

To record *why* a revision happened, set the `kubernetes.io/change-cause` annotation; it appears in `rollout history`.

3.4 Release strategies

Strategy	How it works	Pros / cons
Rolling update	Replace pods gradually (built into Deployment)	Zero downtime, no extra cost; both versions live briefly
Recreate	Stop all old, then start new	Simple; causes downtime
Blue-green	Run full v1 (blue) and v2 (green); switch the Service selector at once	Instant switch and rollback; needs double resources
Canary	Send a small share of traffic (e.g. 10%) to v2, watch metrics, then increase	Lowest risk; needs traffic splitting (replica ratio, Ingress, service mesh, Argo Rollouts)

Everyday example
A canary release is like a restaurant offering a new dish to a few tables first. If those diners love it, it goes on the main menu; if not, only a few people were affected. The name comes from coal miners who carried canaries to detect gas early.

3.5 Live lab: deploy, update, break, roll back

PowerShell - part 1

```
kubectl apply -f deploy.yaml
kubectl get deploy,rs,pods -l app=web

# scale
kubectl scale deployment web --replicas=5
kubectl get pods -l app=web

# rolling update - watch in a second window: kubectl get pods -l app=web -w
kubectl set image deployment/web nginx=nginx:1.27
kubectl annotate deployment/web kubernetes.io/change-cause="upgrade to nginx 1.27"
kubectl rollout status deployment/web
kubectl get rs -l app=web           # old RS has 0 pods, new RS has 5
```

PowerShell - part 2

```
# release a broken version
kubectl set image deployment/web nginx=nginx:9.99-broken
kubectl annotate deployment/web kubernetes.io/change-cause="bad release" --overwrite
kubectl get pods -l app=web         # new pods: ImagePullBackOff, old pods still serve!
kubectl rollout status deployment/web --timeout=30s

# roll back
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
kubectl get pods -l app=web         # all healthy again on nginx:1.27
kubectl describe deployment web | Select-String Image
```

Notice that during the broken release, users were **not** affected: maxUnavailable kept the old pods serving until new ones became ready - which never happened.

Try it yourself
- Change `replicas` in deploy.yaml and run `kubectl apply -f deploy.yaml` again - this is the declarative way to scale.
- Try a simple blue-green: create `web-blue` and `web-green` Deployments with labels `version: blue/green`, then switch a Service selector between them after Class 4.

Common mistake

- Using `kubectl set image` in production and forgetting to update the YAML in Git - the next `apply` reverts your change. Keep Git as the source of truth.
- Thinking `kubectl scale` changes the history. Only pod template changes create revisions.

Class 3 recap

- ReplicaSet keeps N pods; Deployment manages ReplicaSets.
- Rolling update tuned by maxSurge / maxUnavailable.
- Rollback = scale an old ReplicaSet back up (`rollout undo`).
- Blue-green switches all traffic; canary shifts a little at a time.

Class 3 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a ReplicaSet?

A controller that ensures a specified number of pod replicas matching a label selector are running. It creates pods from its template when there are too few and deletes them when there are too many.

Q2. What is a Deployment?

A higher-level object that manages ReplicaSets to provide declarative updates, rolling updates, rollback, scaling and pause/resume for stateless applications.

Q3. What is the difference between a ReplicaSet and a Deployment?

A ReplicaSet only maintains a pod count. A Deployment manages ReplicaSets and adds versioned rollouts, history and rollback. You normally create Deployments, not ReplicaSets.

Q4. What was a ReplicationController?

The older predecessor of ReplicaSet. It supported only equality-based selectors; ReplicaSet adds set-based selectors. It is considered legacy.

Q5. How does a Deployment perform a rolling update?

When the pod template changes, it creates a new ReplicaSet and scales it up while scaling the old one down, respecting maxSurge and maxUnavailable and waiting for new pods to become ready.

Q6. Explain maxSurge and maxUnavailable.

maxSurge is how many pods above the desired count may exist during an update; maxUnavailable is how many may be unavailable. Both default to 25%. maxSurge 1 / maxUnavailable 0 means one extra pod at a time and never fewer ready pods than desired.

Q7. What is the Recreate strategy and when would you use it?

It terminates all old pods before creating new ones, causing downtime. Use it when two versions cannot run simultaneously, such as incompatible schema changes or a single-writer volume.

Q8. How do you roll back a Deployment?

`kubectl rollout undo deployment/<name>` for the previous revision or `--to-revision=N` for a specific one. Kubernetes scales the old ReplicaSet back up.

Q9. How do you check rollout status and history?

`kubectl rollout status deployment/<name>` and `kubectl rollout history deployment/<name>`, with `--revision=N` for details of one revision.

Q10. What triggers a new revision?

Only changes to the pod template (`spec.template`) such as image, env, labels or resources. Scaling replicas does not create a revision.

Q11. What is revisionHistoryLimit?

The number of old ReplicaSets kept for rollback (default 10). Setting it to 0 disables rollback.

Q12. How do you scale a Deployment?

Imperatively with `kubectl scale deployment/<name> --replicas=N`, declaratively by changing `replicas` and running `kubectl apply`, or automatically with a HorizontalPodAutoscaler.

Q13. How do you restart all pods of a Deployment without changing the image?

`kubectl rollout restart deployment/<name>`. It adds a timestamp annotation to the template, triggering a rolling restart - useful after changing a ConfigMap used as env variables.

Q14. What happens if you deploy an image that does not exist?

New pods go into ErrImagePull/ImagePullBackOff and never become ready. With a sensible maxUnavailable the old pods keep serving traffic; the rollout stalls until you fix or undo it.

Q15. What is progressDeadlineSeconds?

How long a Deployment may go without progress before it is reported as failed (default 600 seconds). CI/CD pipelines use this through `kubectl rollout status` to detect bad releases.

Q16. Explain blue-green deployment in Kubernetes.

Run two full environments - blue (current) and green (new). Test green, then switch the Service selector from blue to green so all traffic moves at once. Rollback is switching back. It needs twice the resources.

Q17. Explain canary deployment.

Release the new version to a small share of users first, monitor errors and latency, and increase gradually. In plain Kubernetes this can be done with replica ratios behind one Service; finer control uses Ingress weights, a service mesh, or Argo Rollouts / Flagger.

Q18. Why must the Deployment selector match the template labels?

The selector tells the ReplicaSet which pods it owns. If template labels did not match, created pods would not be counted and the controller would create pods forever, so the API rejects it.

Q19. What does kubectl rollout pause do?

It stops the Deployment from rolling out template changes, letting you make several edits and then `rollout resume` to apply them as a single rollout.

Q20. Are Deployments suitable for databases?

Usually not. Deployment pods are interchangeable with random names and shared storage. Databases need stable identity and per-pod storage, which StatefulSets provide (Class 7).